PhyloResistance AI – Documentação de Implementação

Predição de **Resistência Antimicrobiana (AMR)** e mapeamento de mutações genômicas por **Machine Learning Explicável (XAI)**, a partir de genomas bacterianos. Documento técnico completo para implementar o projeto de ponta a ponta, com rigor científico e foco em reprodutibilidade.

1. Resumo e perguntas científicas

Objetivo: dado o genoma de um isolado bacteriano, prever sua resistência a antibióticos específicos e indicar **quais evidências genômicas** (genes de resistência e mutações) sustentam cada predição, expondo o resultado por uma API.

Perguntas científicas:

ID	Pergunta	Tipo
Q1	Um modelo de ML sobre atributos genômicos prevê resistência melhor do que a detecção de genes e mutações **já conhecidos** (CARD/AMRFinderPlus)?	Preditiva (principal)
Q2	O desempenho se mantém em **linhagens, anos e países não vistos** no treino?	Generalização
Q3	As explicações do modelo (SHAP) recuperam os determinantes **biologicamente conhecidos** e sugerem candidatos novos?	Interpretabilidade
Q4	Quanta incerteza o modelo carrega, e ele consegue **abster-se** nos casos duvidosos?	Confiabilidade

2. Revisão crítica da ideia original (ajustes recomendados)

A proposta inicial é forte, mas alguns pontos precisam de ajuste para o resultado ser **cientificamente válido**. Estes ajustes já estão incorporados nas seções seguintes.

Ponto da ideia original	Risco	Ajuste adotado
K-mers curtos (k = 6 ou 8) como frequências	Um genoma de milhões de bases contém praticamente todos os k-mers curtos; a frequência mede **composição**, não mutações específicas. Sem posição precisa, o SHAP não consegue apontar "gene X, posição Y"	Usar **k-mers longos (k = 21 a 31) em presença/ausência**, mais **genes AMR** e **mutações pontuais** como atributos. Manter k curto apenas como **controle negativo** (ablation)
Explicar "mutação X na posição Y" via SHAP em k-mers	K-mers não têm coordenada própria; SHAP com atributos correlacionados é instável	**Mapear k-mers importantes para loci** (alinhamento a referência e anotação) e **agregar SHAP por gene/mutação**; validar contra o CARD
"Sequências brutas (FASTA/FASTQ)"	FASTQ exige trimagem e montagem (minutos); FASTA é genoma já montado	MVP aceita **FASTA montado**; FASTQ entra como extensão com **pipeline de montagem**
"Instantaneamente"	Montagem, k-mers e anotação têm custo	Definir meta de latência mensurável (seção 9) e usar **processamento assíncrono**
Divisão treino/teste não especificada	Bactérias são **clonais**: isolados parecidos em treino e teste inflam muito a métrica (confusão filogenética)	**Validação por grupo filogenético**, desduplicação e validação temporal/geográfica
Sem baseline definido	O ML pode apenas reaprender o que o CARD já detecta	**Baseline por regras** (genes e mutações conhecidos) obrigatório
"Vetor de resistência para múltiplos fármacos"	Classes e padrões de rotulagem variam por fármaco e espécie	Um **modelo por par espécie–antibiótico**, com extensão multi-saída opcional
Rótulos de fenótipo de várias fontes	Mistura de métodos de laboratório e padrões (CLSI, EUCAST) gera ruído	**Curadoria padronizada de rótulos** (seção 3.3)

3. Dados

3.1 Fontes

Fonte	Conteúdo	Uso no projeto
BV-BRC (Bacterial and Viral Bioinformatics Resource Center)	Genomas bacterianos e **fenótipos de resistência** com metadados (método, padrão, valor de MIC, país, ano)	Fonte principal de genomas e rótulos
CARD (Comprehensive Antibiotic Resistance Database)	Ontologia de determinantes de resistência, genes e mutações, ferramenta RGI	Anotação de genes AMR, **baseline** e **verdade biológica** para validar o SHAP
NCBI RefSeq / GenBank	Genomas e sequências de referência	Genomas de referência por espécie, anotação
NCBI AMRFinderPlus (banco)	Genes e mutações de resistência curados	Anotação complementar e baseline
NCBI Pathogen Detection (dados de AST) (opcional)	Isolados com testes de sensibilidade	**Validação externa**

> Disponibilidade, formatos, termos de uso e versões mudam com o tempo. Na Fase 0, registre **data, versão, origem e hash** de cada banco e arquivo baixado, e confirme os termos de uso.

3.2 Escopo piloto

Comece pequeno e escale depois que o pipeline estiver validado.

Espécie	Fármacos do piloto	Motivo
Escherichia coli	ciprofloxacina, gentamicina, cefuroxima (ou ceftriaxona)	Muitos genomas públicos; mecanismos conhecidos (mutações em `gyrA` / `parC`, β-lactamases, enzimas modificadoras de aminoglicosídeos)

Espécie	Fármacos do piloto	Motivo
Klebsiella pneumoniae	ciprofloxacina, gentamicina, ceftriaxona (fase 2)	Patógeno prioritário; extensão natural
Staphylococcus aureus	oxacilina (MRSA), ciprofloxacina (fase 3)	Mecanismo bem definido (`mecA`), ótimo caso de controle

Tamanho de referência (heurística de estudo): alvo de pelo menos 1.000 genomas por par espécie–fármaco, com pelo menos algumas centenas na classe minoritária. Pares com poucos casos resistentes devem ser descartados ou tratados como exploratórios.

3.3 Curadoria de rótulos (crítica para a qualidade)

Regra	Ação
Padronizar a categoria	Converter para **Resistente (R)**, **Sensível (S)** e tratar "Intermediário (I)" de forma explícita: excluir do treino principal ou agrupar com S em análise de sensibilidade
Registrar método e padrão	Guardar método (microdiluição, disco-difusão, etc.), padrão (CLSI ou EUCAST) e versão do ponto de corte
Preferir MIC quando existir	Recalcular a categoria a partir do MIC com um **único ponto de corte** documentado, em vez de misturar interpretações
Conflitos	Isolado com rótulos contraditórios para o mesmo fármaco: excluir ou resolver por regra documentada
Funil de exclusão	Registrar quantos registros saem em cada etapa e por quê

3.4 Controle de qualidade dos genomas

Verificação	Ferramenta sugerida	Critério
Estatísticas de montagem	QUAST	Tamanho total, nº de contigs e N50 dentro da faixa da espécie

Verificação	Ferramenta sugerida	Critério
Completude e contaminação	CheckM2	Limiares documentados (por exemplo, alta completude e baixa contaminação)
Confirmação da espécie	Mash, FastANI/skani ou GTDB-Tk	Espécie declarada igual à inferida
Desduplicação	Mash/skani	Remover genomas praticamente idênticos (limiar definido no plano de análise)

4. Arquitetura do pipeline

```
  BV-BRC / NCBI / CARD
        |  download versionado
        ▼
  [1] Ingestão e curadoria de rótulos
        ▼
  [2] QC dos genomas + confirmação de espécie + desduplicação
        ▼
  [3] Estrutura filogenética (distâncias Mash → clusters / ST)
        ▼
  [4] Atributos:  F1 genes AMR  •  F2 mutações pontuais  •  F3 k-mers
(k=21..31)
        ▼
  [5] Divisão por grupo (treino / validação / teste temporal e geográfico)
        ▼
  [6] Modelagem: baselines → regressão regularizada → XGBoost/LightGBM
        ▼
  [7] Calibração + incerteza (abstensão)
        ▼
  [8] Explicabilidade: SHAP → mapeamento k-mer → locus → relatório
        ▼
  [9] Serviço FastAPI (jobs assíncronos) + relatório JSON/HTML
```

O pipeline bioinformático (etapas 1 a 4) é orquestrado por **Snakemake ou Nextflow**, com containers e versões travadas.

5. Engenharia de atributos

5.1 Três conjuntos de atributos

Conjunto	Descrição	Ferramentas	Observações
F1 – Genes de resistência	Matriz binária de presença/ausência de genes AMR (por identidade e cobertura mínimas)	AMRFinderPlus, RGI (CARD), ResFinder	Base do baseline por regras
F2 – Mutações pontuais	Binários para alterações conhecidas (por exemplo, `gyrA` S83L) e SNPs contra a referência	AMRFinderPlus (modo por organismo), Snippy ou MUMmer	Essencial para quinolonas
F3 – K-mers	Presença/ausência de k-mers longos (k = 21 a 31), filtrados	KMC3, Jellyfish ou kmtricks	Captura variantes não catalogadas; matriz **esparsa**

5.2 Contagem e filtragem de k-mers

```
# k-mers canônicos (k=31) de um genoma montado
kmc -k31 -ci1 -fm -t8 genoma.fasta saida/genoma tmp/
kmc_tools transform saida/genoma dump saida/genoma.txt
```

- **Presença/ausência** (e não frequência), para evitar depender da cobertura e da montagem.
- **Filtro de frequência:** descartar k-mers presentes em quase todos os genomas (não informativos) ou em pouquíssimos (ruído). O filtro é definido **no conjunto de treino**.
- **Redução supervisionada** (por exemplo, teste qui-quadrado ou informação mútua) feita **dentro de cada dobra**, nunca no conjunto completo.
- **Representação:** `scipy.sparse` ou Parquet, com mapa de k-mer para identificador. A matriz densa não cabe em memória (milhões de k-mers únicos).
- **Agrupar k-mers equivalentes** (que sempre aparecem juntos) em unidades "unitigs" ou "padrões" reduz dimensão e melhora a interpretação.

5.3 Metadados

País, ano, ST (tipo de sequência) e linhagem **não entram como atributos** do modelo principal. Servem para estratificar a análise, construir splits e investigar vieses. Um modelo que usa metadados pode estar apenas aprendendo a epidemiologia local.

6. Modelagem

6.1 Escada de modelos (do mais simples ao mais complexo)

Nível	Modelo	Atributos	Papel
B0	Classe majoritária	–	Piso trivial
B1	**Regra: resistente se houver determinante conhecido**	F1 + F2	**Baseline científico obrigatório**
B2	Regressão logística regularizada (L1 ou elastic net)	F1 + F2	Linear e interpretável
B3	XGBoost ou LightGBM	F1 + F2	Modelo principal curado
B4	XGBoost ou LightGBM	F1 + F2 + F3	Captura variantes novas
B5	Linear/esparso em k-mers (por exemplo, regressão regularizada)	F3	Comparação com a literatura
B6 (opcional)	CNN ou transformer em sequências	Sequência	Extensão; tende a exigir muito mais dados
Controle	XGBoost com frequência de k-mers curtos (k = 6 a 8)	Composição	**Controle negativo** que mostra por que k curto é insuficiente

Pergunta decisiva (Q1): B3 e B4 superam B1 em grupos filogenéticos **não vistos**? Se não superarem, o resultado é válido e deve ser reportado como tal.

6.2 Treinamento

```
from sklearn.model_selection import StratifiedGroupKFold
import xgboost as xgb

cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)

for treino, teste in cv.split(X, y, groups=cluster_filogenetico):
    # filtro de k-mers, seleção de atributos e ajuste de hiperparâmetros
    # usam APENAS os dados de treino desta dobra
    modelo = xgb.XGBClassifier(
        n_estimators=400, max_depth=4, learning_rate=0.05,
        subsample=0.8, colsample_bytree=0.5,
        scale_pos_weight=peso_classe, eval_metric="aucpr",
    )
    modelo.fit(X[treino], y[treino])
    prob = modelo.predict_proba(X[teste])[:, 1]
```

- **Ajuste de hiperparâmetros** com Optuna em **validação aninhada por grupo**.
- **Desbalanceamento:** pesos de classe; evitar *oversampling* sintético que cria genomas biologicamente irreais.
- **Calibração:** isotônica ou Platt, ajustada em dados de calibração separados.
- **Incerteza e abstenção:** predição conformal (por exemplo, MAPIE) para devolver "indeterminado" quando a confiança for baixa, reduzindo erros graves.
- **MIC (opcional):** regressão de log2(MIC) com **concordância essencial** (diferença de até uma diluição).

7. Validação

7.1 Estratégias de divisão

Estratégia	Como	Uso
Aleatória	Split comum	**Apenas para demonstrar o vazamento** e reportar como limite superior enganoso
Por grupo filogenético	Clusters por distância Mash/ANI ou por ST; todos os membros de um cluster no mesmo lado	**Validação principal**
Leave-one-lineage-out	Deixar uma linhagem (por exemplo, um ST) inteira de fora	Teste de generalização entre linhagens

Estratégia	Como	Uso
Temporal	Treinar em anos antigos, testar nos mais recentes	Simula uso real
Geográfica	Treinar em alguns países, testar em outros	Robustez à epidemiologia local
Externa	Outra fonte de dados (por exemplo, Pathogen Detection)	Prova independente

7.2 Métricas

Métrica	Uso	Observação
AUROC e AUPRC	Discriminação	AUPRC é mais informativa com classes desbalanceadas
Sensibilidade, especificidade e acurácia balanceada	Desempenho por classe	Reportar no ponto de corte escolhido
VME (*very major error*)	Predizer S quando o isolado é R	O erro mais perigoso clinicamente
ME (*major error*)	Predizer R quando o isolado é S	Leva ao uso de antibiótico desnecessariamente restrito
Concordância categórica e essencial	Padrão de avaliação de testes de sensibilidade (ISO 20776-2)	Confirmar critérios vigentes antes de comparar
Calibração (Brier, curva de calibração)	Qualidade das probabilidades	Obrigatória
Cobertura e taxa de abstenção	Predição conformal	Mostrar troca entre cobertura e erro

Estatística: intervalos de confiança por *bootstrap* **em blocos (por cluster)**, e não por isolado, e comparação pareada entre modelos e baseline.

8. Explicabilidade (XAI)

8.1 Plano

1. Calcular **TreeSHAP** para o modelo de árvores (rápido e exato para XGBoost e LightGBM).
2. **Mapear k-mers de maior impacto** para loci: alinhar os k-mers (ou os unitigs) ao genoma de referência e à anotação (Bakta ou Prokka) com BLAST ou minimap2.
3. **Agregar** a contribuição por **gene ou mutação**, pois k-mers vizinhos são redundantes e dividem o mesmo sinal.
4. Cruzar com o **CARD/AMRFinderPlus** e classificar cada evidência como *conhecida* ou *candidata*.
5. Gerar o **relatório por isolado**: lista de evidências, direção e magnitude da contribuição, e fonte.

```
import shap

explicador = shap.TreeExplainer(modelo)
valores = explicador.shap_values(X_teste)
# agregar por locus (gene ou mutação) antes de reportar
contribuicao_por_locus = agregar_por_locus(valores, mapa_atributo_para_locus)
```

8.2 Como avaliar as explicações (e não só apresentá-las)

Teste	Pergunta	Critério
Recuperação de determinantes conhecidos	Entre os atributos mais importantes, quantos são determinantes catalogados para aquele fármaco?	Proporção dos *k* principais presente no CARD
Estabilidade	As explicações se repetem em dobras e *bootstraps* diferentes?	Correlação de ranking entre repetições
Plausibilidade biológica	O locus faz sentido para o mecanismo (alvo do fármaco, bomba de efluxo, enzima)?	Revisão por especialista
Comparação com GWAS bacteriano	Os candidatos coincidem com associações de ferramentas como `pyseer`?	Concordância parcial esperada

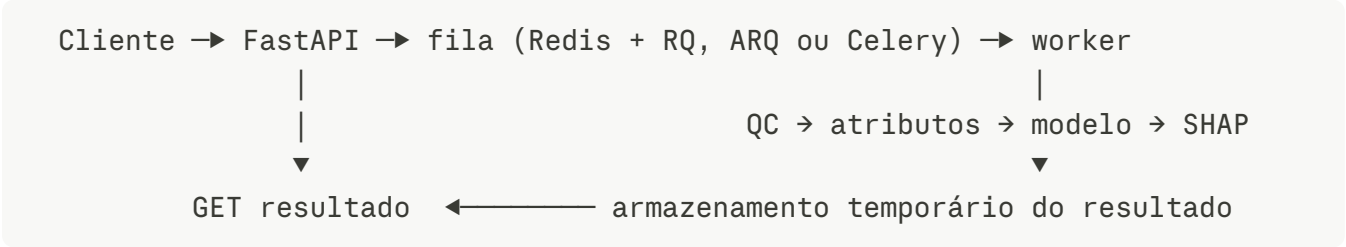
Teste	Pergunta	Critério
Teste de remoção	Remover o locus principal reduz a probabilidade como esperado?	Queda coerente

8.3 Cuidados

- **SHAP mede contribuição ao modelo, não causalidade biológica.** Candidatos novos são **hipóteses** que exigiriam validação experimental.
- **Confusão filogenética:** atributos que marcam uma linhagem podem parecer "causais" só porque a linhagem é resistente. A validação por grupo e a comparação com GWAS corrigido por estrutura populacional reduzem esse risco.
- **Atributos correlacionados** dividem a importância de forma arbitrária; por isso a agregação por locus.

9. Serviço de predição (FastAPI)

9.1 Arquitetura



A predição é **assíncrona**: o envio devolve um `job_id`, e o resultado é consultado depois. Isso evita requisições longas e permite escalar *workers*.

9.2 Endpoints

Método e rota	Função
`POST /v1/predicoes`	Envia o FASTA (e opcionalmente espécie e lista de fármacos); retorna `202` com `job_id`
`GET /v1/predicoes/{job_id}`	Estado do job e, quando pronto, o resultado
`GET /v1/predicoes/{job_id}/relatorio`	Relatório em HTML ou PDF
`GET /v1/modelos`	Modelos disponíveis, versões e *model cards*

Método e rota	Função
`GET /health` e `GET /ready`	Vivacidade e prontidão (modelos e bancos carregados)

```
@app.post("/v1/predicoes", status_code=202)
async def criar_predicao(arquivo: UploadFile, especie: str | None = None,
                         farmacos: list[str] | None = None):
    caminho = await salvar_temporario(arquivo, limite_mb=LIMITE_MB)
    validar_fasta(caminho)   # formato, alfabeto (ACGTN), nº de contigs,
tamanho
    job = fila.enqueue(processar_genoma, caminho, especie, farmacos)
    return {"job_id": job.id, "status": "em_fila"}
```

9.3 Exemplo de resposta

```
{
  "job_id": "a1b2c3",
  "especie": "Escherichia coli",
  "qc": {"tamanho_pb": 5120345, "contigs": 142, "n50": 120345, "aprovado":
true},
  "predicoes": [
    {
      "farmaco": "ciprofloxacina",
      "probabilidade_resistencia": 0.93,
      "classe": "resistente",
      "confianca": "alta",
      "evidencias": [
        {"tipo": "mutacao_pontual", "gene": "gyrA", "alteracao": "S83L",
         "fonte": "AMRFinderPlus", "contribuicao_shap": 0.41, "status":
"conhecida"}
      ]
    }
  ],
  "versao_modelo": "ecoli-cip-1.2.0",
  "versoes_bancos": {"amrfinderplus": "x.y.z", "card": "x.y.z"},
  "aviso": "Uso em pesquisa. Não substitui o antibiograma."
}
```

9.4 Requisitos do serviço

Requisito	Especificação
Validação de entrada	Tamanho máximo, formato FASTA, alfabeto, número de contigs, rejeitar arquivos que não são genomas bacterianos
QC no serviço	Mesmos critérios do treino; **rejeitar** ou **sinalizar** genomas fora do domínio (espécie não suportada, baixa qualidade)
Domínio de aplicação	O modelo só é válido para as espécies e fármacos treinados; fora disso, retornar "não suportado"
Latência	Meta a medir: p95 para FASTA montado dentro de um limite definido (por exemplo, 1 a 2 minutos); FASTQ inclui a montagem e leva mais
Segurança	Chave de API, *rate limit*, HTTPS, isolamento de containers, arquivos temporários apagados
Privacidade	Retenção mínima, sem armazenar sequências por padrão, e política clara de dados
Reprodutibilidade	Containers com as mesmas versões de ferramentas e bancos do treino; versões registradas em cada resposta
Observabilidade	Logs estruturados, métricas de fila e latência, monitoramento de *drift* (distribuição de espécies, QC, probabilidades)
Documentação	OpenAPI automática e *model card* por modelo

10. Estrutura do repositório

```
phyloresistance/
├── README.md
├── pyproject.toml                # dependências Python travadas
├── Makefile
├── docs/
|   ├── plano_de_analise.md       # hipóteses, métricas, splits (antes de
ver o teste)
|   ├── curadoria_de_rotulos.md
|   ├── model_card_ecoli_cip.md
```

```
|   └── decisoes/                    # ADRs
├── workflow/                        # Snakemake ou Nextflow
|   ├── rules/                       # ingest, qc, dedup, features, kmers
|   └── envs/                        # ambientes conda / containers
├── data/
|   ├── raw/                         # nunca editado; versionado por hash
|   ├── interim/
|   └── processed/                   # matrizes esparsas e tabelas de rótulos
├── src/phyloresistance/
|   ├── dados/                       # BV-BRC, curadoria, contratos de dados
|   ├── genomica/                    # QC, k-mers, anotação, mutações
|   ├── filogenia/                   # distâncias, clusters, splits
|   ├── modelos/                     # baselines, regressão, XGBoost,
calibração
|   ├── validacao/                   # métricas, VME e ME, bootstrap em blocos
|   ├── explicabilidade/             # SHAP, mapeamento para loci, relatório
|   └── api/                         # FastAPI, workers, esquemas
├── notebooks/                       # exploração e narrativa
├── tests/                           # unitários, contratos, anti-vazamento,
API
├── docker/                          # imagens da API e do worker
└── reports/                         # relatório final e figuras
```

11. Regras de rigor científico (obrigatórias)

ID	Regra	Verificação
REG-01	Desempenho reportado **somente** com divisão por grupo filogenético (e, quando possível, temporal e geográfica)	Teste automatizado do split
REG-02	**Desduplicação** de genomas quase idênticos antes de dividir	Relatório de desduplicação
REG-03	Filtro de k-mers, seleção de atributos e ajuste de hiperparâmetros apenas com dados de treino de cada dobra	Teste anti-vazamento
REG-04	**Baseline por regras (B1)** obrigatório; o ML só é considerado melhor se superar B1 com intervalo de confiança	Relatório
REG-05	Reportar **VME e ME** por fármaco, além de AUROC e AUPRC	Relatório

ID	Regra	Verificação
REG-06	Rótulos padronizados (método, padrão, ponto de corte, tratamento de "I") e funil de exclusão documentado	Documento de curadoria
REG-07	**QC dos genomas** e confirmação da espécie antes de qualquer uso	Gates no pipeline
REG-08	Metadados (país, ano, ST) não são atributos do modelo principal	Revisão + teste
REG-09	Validação **externa, temporal ou geográfica** antes de afirmar generalização	Relatório
REG-10	SHAP **agregado por locus** e verificado contra o CARD; linguagem de associação, nunca de causalidade	Revisão
REG-11	Calibração e incerteza reportadas; o serviço pode abster-se	Relatório e testes
REG-12	Plano de análise registrado antes de abrir o teste final, que é aberto **uma única vez**	Commit com data
REG-13	Reprodutibilidade: workflow com containers, versões de ferramentas e bancos, seeds e hashes	CI
REG-14	Toda predição da API registra a versão do modelo e dos bancos	Teste de API
REG-15	O projeto é de **pesquisa**; nenhuma alegação de uso clínico sem validação e regulação adequadas	Revisão

12. Stack tecnológica

Área	Ferramentas
Orquestração	Snakemake ou Nextflow; containers Docker/Apptainer; Bioconda
Dados e formatos	`pandas` ou `polars`, `pyarrow` (Parquet), `scipy.sparse`, `biopython`
Genoma e QC	QUAST, CheckM2, Mash, skani ou FastANI, GTDB-Tk (opcional)

Área	Ferramentas
Montagem (extensão FASTQ)	fastp, Shovill/SPAdes
Anotação AMR	AMRFinderPlus, RGI (CARD), ResFinder
Variantes	Snippy ou MUMmer, `bcftools`
K-mers	KMC3, Jellyfish, kmtricks
Filogenia	Distâncias Mash, FastTree ou IQ-TREE (opcional), `scipy` para clustering
GWAS bacteriano (comparação)	`pyseer`
Machine learning	`scikit-learn`, `xgboost`, `lightgbm`, `optuna`, `MAPIE`
Explicabilidade	`shap`, BLAST ou minimap2, Bakta ou Prokka
Experimentos	MLflow (métricas, artefatos e registro de modelos)
API	FastAPI, Pydantic, Redis com RQ ou ARQ, Uvicorn
Qualidade de código	`pytest`, `ruff`, `mypy`, `pre-commit`, GitHub Actions
Versionamento de dados	DVC
Relatório	Quarto ou Jupyter Book

12.1 Recursos computacionais (ordem de grandeza)

- **Piloto (2 mil a 5 mil genomas):** estação de trabalho ou VM com 8 a 16 núcleos, 32 a 64 GB de RAM e algumas dezenas de GB de disco (os genomas montados ocupam cerca de 5 MB cada, e as matrizes de k-mers filtradas, bem menos).
- Contagem de k-mers e anotação AMR são **paralelizáveis por genoma**; o gargalo costuma ser a anotação e a montagem, não o ML.
- Meca a latência real de cada etapa na Fase 5 antes de prometer tempos.

13. Roteiro em fases

Fase	Entrega	Critério de aceite	Duração sugerida
0 – Fundação	Repositório, ambiente, CI, plano de análise, escolha de espécie e fármacos	Plano versionado antes de qualquer modelo	1 semana
1 – Dados e curadoria	Download versionado, rótulos padronizados, QC, desduplicação, funil de exclusão	Tabela final de rótulos e genomas aprovados e documentada	2 semanas
2 – Atributos e baseline	F1, F2, F3, estrutura filogenética, splits por grupo, **baseline B1**	Baseline reproduz resultados e teste anti-vazamento verde	2 semanas
3 – Modelos e validação	B2 a B5, controle de k curto, calibração, VME e ME, validação temporal e geográfica	Comparação com IC e conclusão honesta sobre Q1 e Q2	2 a 3 semanas
4 – Explicabilidade	SHAP, mapeamento para loci, avaliação das explicações, relatório por isolado	Testes da seção 8.2 concluídos	2 semanas
5 – Serviço	API assíncrona, containers, QC no serviço, model card, testes de carga	Predição ponta a ponta reproduz o resultado do pipeline	1 a 2 semanas
6 – Extensões	Novas espécies e fármacos, FASTQ, MIC, rede neural (opcional)	Ganho medido em validação externa	Conforme interesse
7 – Comunicação	Relatório final, demonstração, README e vídeo curto	Qualquer pessoa reproduz os resultados com um comando	1 semana

14. Riscos e armadilhas

Risco	Prob.	Impacto	Mitigação
Vazamento filogenético inflando as métricas	Alta	Alto	Divisão por grupo, desduplicação, validação temporal e externa
Rótulos ruidosos ou heterogêneos	Alta	Alto	Curadoria padronizada e análise de sensibilidade
ML não supera o baseline de regras	Média	Médio	Resultado válido; explorar casos em que o ML complementa as regras (variantes não catalogadas)
Explosão dimensional dos k-mers (memória e tempo)	Alta	Médio	Filtros, representação esparsa, unitigs, seleção dentro da dobra
Explicações instáveis ou espurias	Alta	Médio	Agregação por locus, estabilidade, comparação com CARD e GWAS
Qualidade da montagem afetando atributos	Média	Médio	QC rigoroso; usar presença/ausência; filtrar genomas fragmentados
Desbalanceamento extremo em alguns fármacos	Média	Médio	Pesos, métricas adequadas, descartar pares sem dados suficientes
Mudança de bancos e ferramentas entre treino e serviço	Média	Alto	Versões travadas, containers, registro das versões em cada predição
Drift (novas linhagens e mutações)	Média	Alto	Monitoramento, reavaliação periódica, abstenção
Uso indevido do serviço como decisão clínica	Média	Alto	Avisos explícitos, restrição de domínio e *model card*
Sobreajuste ao teste por múltiplas tentativas	Média	Alto	Teste final aberto uma vez; plano pré-registrado

15. Ética, regulação e uso responsável

- O projeto é de **pesquisa e demonstração técnica**. Não substitui o antibiograma nem deve orientar a prescrição.

- Um sistema que apoie decisão clínica pode ser enquadrado como **software como dispositivo médico** (por exemplo, pela ANVISA no Brasil, e por regras equivalentes em outras regiões), exigindo validação e regularização. Isso está fora do escopo deste projeto.
- Os dados são genomas bacterianos públicos, com baixo risco de privacidade, mas os **metadados** (local, data, hospedeiro) devem ser tratados com cuidado e respeitar os termos de cada fonte.
- Comunique sempre **limitações**: domínio de espécies, fármacos, período e regiões dos dados, e a incerteza das predições.
- Cite as fontes e licenças das bases e ferramentas utilizadas.

16. Entregáveis e portfólio

- Repositório reproduzível com `make` ou comando único do workflow.
- **Plano de análise** e **documento de curadoria de rótulos**.
- Relatório (Quarto) com perguntas Q1 a Q4 respondidas, métricas com intervalos, VME e ME, e seção de limitações.
- **Model card** por modelo e **datasheet** do conjunto de dados.
- API containerizada com documentação OpenAPI e exemplo de relatório por isolado.
- Notebook de explicabilidade com os casos de recuperação de determinantes conhecidos e candidatos novos.
- Demonstração em vídeo curto.

Critérios de sucesso (metas de estudo, não garantias): pipeline reproduzível de ponta a ponta; comparação honesta com o baseline por regras em grupos não vistos; VME e ME reportados por fármaco; explicações validadas contra o CARD; API que devolve relatório com versões e avisos.

17. Referências

- Alcock, B. et al. — *CARD* (Comprehensive Antibiotic Resistance Database), *Nucleic Acids Research*.
- Olson, R. et al. — *BV-BRC* (Bacterial and Viral Bioinformatics Resource Center), *Nucleic Acids Research*.
- Feldgarden, M. et al. — *AMRFinderPlus*, *Scientific Reports* (2021).
- Bortolaia, V. et al. — *ResFinder 4.0*, *Journal of Antimicrobial Chemotherapy* (2020).
- Nguyen, M. et al. — modelos de XGBoost para MIC em *Klebsiella pneumoniae*, *Scientific Reports* (2018).

- Moradigaravand, D. et al. — predição de resistência em *E. coli* a partir de dados de pan-genoma, *PLOS Computational Biology* (2018).
- Drouin, A. et al. — fenotipagem preditiva e descoberta de biomarcadores com comparações de genomas sem referência (k-mers), *BMC Genomics* (2016).
- Lundberg, S. e Lee, S.-I. — "A Unified Approach to Interpreting Model Predictions" (SHAP), NeurIPS (2017); Lundberg et al. — TreeSHAP, *Nature Machine Intelligence* (2020).
- Chen, T. e Guestrin, C. — *XGBoost*, KDD (2016).
- Ondov, B. et al. — *Mash*, *Genome Biology* (2016).
- Lees, J. et al. — *pyseer*, *Bioinformatics* (2018).
- Kapoor, S. e Narayanan, A. — "Leakage and the reproducibility crisis in machine-learning-based science", *Patterns* (2023).
- ISO 20776-2 e documentos do CLSI e do EUCAST (pontos de corte e critérios de desempenho de testes de sensibilidade).
- OMS — *Lista de patógenos bacterianos prioritários* e materiais sobre resistência antimicrobiana.

18. Próximo passo

Fechar o **escopo piloto** (espécie e três fármacos, sugestão: *E. coli* com ciprofloxacina, gentamicina e cefuroxima), escrever o **plano de análise** e iniciar a **Fase 0**. A primeira prova de valor é o **baseline por regras (B1)** funcionando com divisão por grupo filogenético, pois ele define o patamar que o modelo de ML precisa superar.